```
_context.Recipes.Where(r=>r.RecipeId == cmd.Id).FirstOrDefault();
```

Using `FindAsync()` or `Find()` is a little more declarative and concise, however.

TIP `Find` is a bit more complicated. `Find` first checks to see whether the entity is already being tracked in EF Core's `DbContext`. If so (because the entity was previously loaded in this request), the entity is returned immediately without calling the database. Using `Find` can obviously be faster if the entity *is* tracked, but it can be slower if you know that the entity isn't being tracked yet.

You may wonder how EF Core knows which columns to update when you call `SaveChangesAsync()`. The simplest approach would be to update every column. If the field hasn't changed, it doesn't matter if you write the same value again. But EF Core is cleverer than that.

EF Core internally tracks the *state* of any entities it loads from the database and creates a snapshot of all the entity's property values so that it can track which ones have changed. When you call `SaveChanges()`, EF Core compares the state of any tracked entities (in this case, the `Recipe` entity) with the tracking snapshot. Any properties that have been changed are included in the `UPDATE` statement sent to the database, and unchanged properties are ignored.

NOTE EF Core provides other mechanisms to track changes, as well as options to disable change tracking. See the documentation or chapter 3 of Jon P. Smith's *Entity Framework Core in Action*, 2nd ed., (Manning, 2021; http://mng.bz/q9PJ) for details. You can view which details the `DbContext` is tracking by accessing